UNIVERSITY OF PISA

Department of Information Engineering

Internet of Things Course

Smart Building Project

Author:
Ivan Brillo

ACADEMIC YEAR 2024/2025

Contents

1 Introduction

1.1 Motivations

The rapid advancement of Internet of Things (IoT) technologies has revolutionized building automation and energy management systems. As environmental concerns grow and energy costs rise, the development of intelligent building energy control systems has become increasingly important. This project presents a comprehensive IoT-based building energy management system designed to address these challenges.
The GitHub repository is present at the following link (deploy branch).

1.2 System Overview

The proposed system consists of three primary components: floor-level sensors and actuators, building-level routers with predictive capabilities for future energy consumption, and a centralized cloud-based management application. The system uses the Constrained Application Protocol (CoAP) to enable efficient communication between IoT devices, allowing real-time monitoring and control of various building subsystems, including air conditioning/heating, automated windows, and battery energy storage systems.

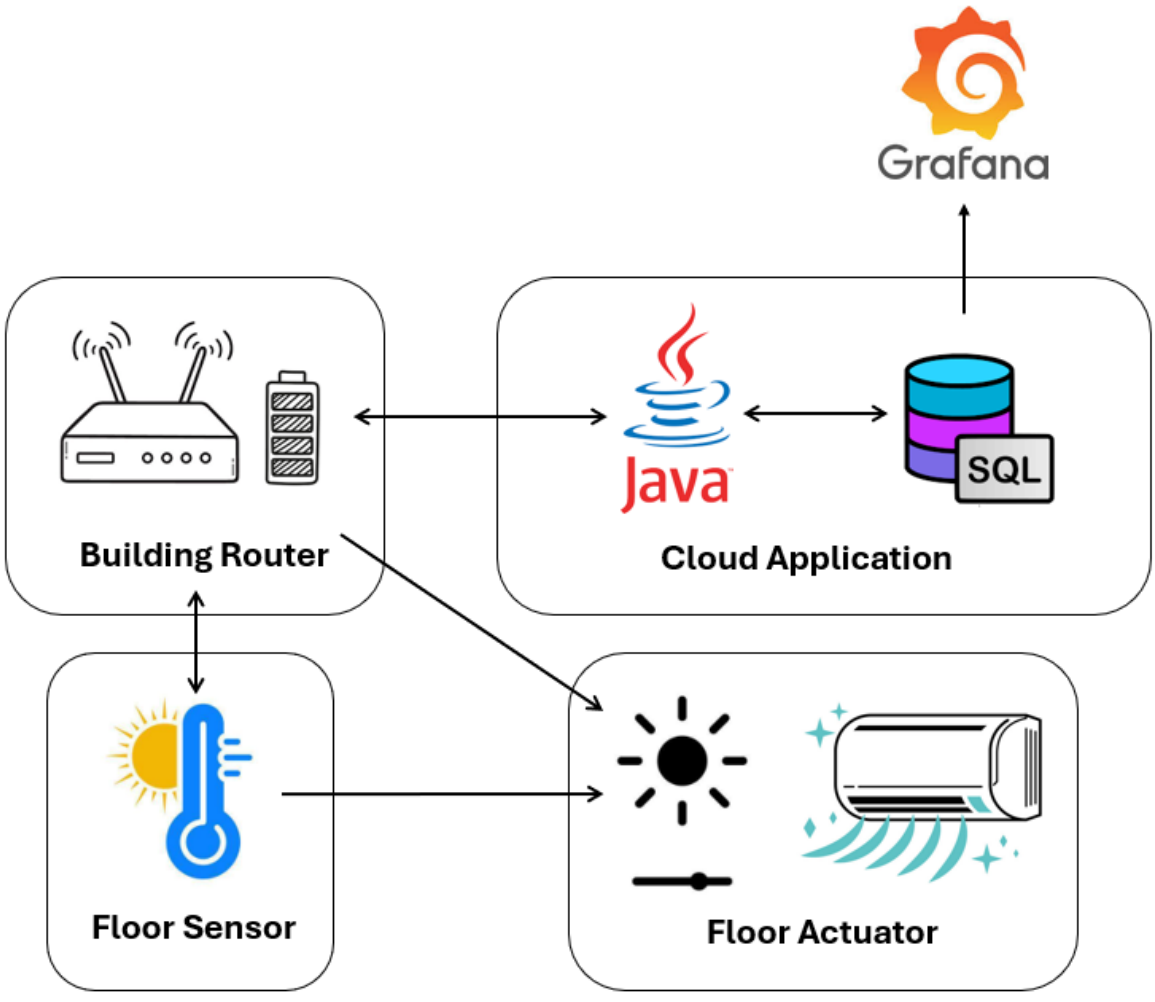

Figure 1.1: System Architecture

At the foundation of the system, floor-level sensor nodes continuously monitor environmental parameters such as temperature and ambient light levels. These sensors support control algorithms that manage actuators responsible for adjusting air conditioning setpoints and operating automated window coverings. The building-level router implements predictive models to forecast energy consumption patterns, enabling proactive energy management strategies across all floor-sensors. It also maintains connectivity with battery management systems, monitors the state-of-charge, and executes client-defined dynamic control policies to optimize energy storage utilization.
The cloud application provides a system overview and enables control of all actuators. An automated system to control the charging of the battery, based on current electricity prices, is run on the cloud application. Data received via CoAP is automatically stored in an SQL database. Additionally, a Grafana interface has been implemented to monitor real-time sensor values.
The whole system has been developed by keeping the number of floors scalable, with minimum changes.

2 CoAP Network Architecture

The Constrained Application Protocol (CoAP) serves as the backbone of the communication infrastructure in this IoT building energy management system. The CoAP observable resources are exploited intensively, in order to avoid data polling and the transmission of useless data (not changed from the previous request), with the consequent waste of energy.

2.1 Building Router as Central Hub

The building router operates as the primary CoAP server, hosting multiple resources and coordinating communication between various floors and cloud application. The resources are accessible via standardized URI paths:

- `/power` - Observable resource that provides real-time power consumption predictions based on machine learning algorithms. It reports also the last active and reactive power readings

```
{"pred": (float active power kW), "last": [(float active power kW), (float
    ↪ reactive power kW)], "v": (int)}
```

- `/battery/soc` - Observable resource that reports current state-of-charge for the building's energy storage system

```
{"soc": (float 0-100%), "v": (int)}
```

- `/battery/setpoint?setpoint=(float charging power kW)` - Accepts battery management commands via POST method. Fist, it validates the correct range $[-10, +10]$ kW and then apply the setpoint selected

- `/energy-modality` - Observable resource that provide energy operational modes. In particular 0 - normal, 1 - light power saving, 2 - heavy power saving. This information is sent as binary data serialization, since it's read only from other C programmed IoT devices. In particular this information is used by floor sensors to regulate the stringency by which AC temperature control is applied. The data is sent only when a change in energy modality is computed by the router

```
typedef struct
{
  int32_t version;
  int8_t modality;
} energy_modality_response_t;
```

The router employs periodic resource triggering mechanisms, with power predictions updated every T_p seconds and battery state-of-charge readings refreshed every T_c seconds. This temporal organization ensures timely data availability while minimizing network congestion and energy consumption.

2.2 Floor-Level Sensor and Actuator Nodes

Floor-level sensors implement CoAP server and client logic. The sensor nodes expose environmental monitoring light and temperature, while actuator nodes, that are CoAP servers, provide control interfaces for windows covering and AC. The actuators can be controlled automatically from the floor sensors,

giving a temperature/light value to maintain or manually via the cloud application. If the former is selected, then a control algorithm is in charge of achieving the selected setpoints, keeping in mind also the energy saving modality for the AC control.
Sensor nodes implement the following resource structure:

- `LIGHT/setpoint/setpoint=(float lumen)` - Light value to keep across the floor, set via POST method
- `TEMP/setpoint/setpoint=(float °C)` - Temperature value to keep across the floor, set via POST method
- `SENSORS/reeding` - Observable resource that reports light and temperature values every T_s seconds if their change is significant with respect to the previous message, in order to save energy.

```
{"light": (float lumen), "temp": (float degC), "v":(int)}
```

Actuator nodes implement the following resource control via POST methods:

- `/AC/setpoint?on=(0|1)&setpoint=(float °C)` - Air conditioning temperature control interface
- `/Window/setpoint?setpoint=(float 0-100%)` - Automated window covering control system

2.3 Cloud Application

The cloud application will subscribe to light/temperature sensors resource for each floor, SOC and power prediction resources. Additionally it has the ability to send the setpoints to both actuator and sensor for each floor and to command the battery setpoint manually, or with an autonomous control algorithm.

2.4 Problems with Float Encoding in nRF52840 Micro-controllers

Unfortunately, all print-related functions, such as `sprintf`, do not work on our dongle. To address this issue in the CoAP communication of floating-point values, we implemented a custom function that encodes a float into a character array, with a maximum precision of two decimal digits.

2.5 Motivation for JSON and Binary Data Encoding

The system uses two main encoding approaches based on communication requirements:
JSON for Cross-Platform Messages: All messages exchanged between C-programmed IoT nodes and the Java cloud application use JSON encoding. Float values are limited to 2 decimal places maximum to optimize bandwidth and processing efficiency.
Binary for IoT-Internal Communication: Power-mode configuration data, used exclusively between IoT nodes, employs C struct serialization for maximum efficiency.

Why Not XML?

XML was deliberately avoided due to IoT resource constraints:

- High memory and CPU overhead for parsing and validation
- Verbose syntax increases network bandwidth usage
- Complex parsing libraries unsuitable for embedded systems

JSON provides the optimal balance of simplicity, efficiency, and cross-platform compatibility for IoT devices and Java cloud application.

2.6 Motivation for CoAP

We used CoAP over MQTT for several reasons:

- CoAP enables one-to-one communication without a broker, reducing complexity, latency, and overhead
- The system continues working without an internet connection; in contrast, if the broker is placed in the cloud, MQTT would experience a service interruption
- CoAP offers superior energy efficiency, as MQTT was not originally designed with IoT devices in mind

2.6.1 CoAP Observable Resources

We extensively utilized observable resources, as they eliminate the need for continuous polling and the transmission of redundant data, thereby reducing energy consumption.
However, this approach can introduce a potential issue: when a device reboots, any existing observe relations another nodes held with it become invalid. To mitigate this, we introduce a timeout mechanism. For each observed resource, if no updates are received for more than T seconds, the node automatically removes and re-establishes the observe relation.

3 Control Logic

3.1 ML Border-Router Prediction

The dataset used to train the ML model is the Household Power Consumption dataset. We selected the global and reactive power measurements and scaled them to the interval $[-1, +1]$. The values were then encoded as `int16`, and a `RandomForestRegressor` was trained using a time window of 5 samples. The trained model was compressed using `emlearn` and exported as a *.h* file. The results of the training are shown below:

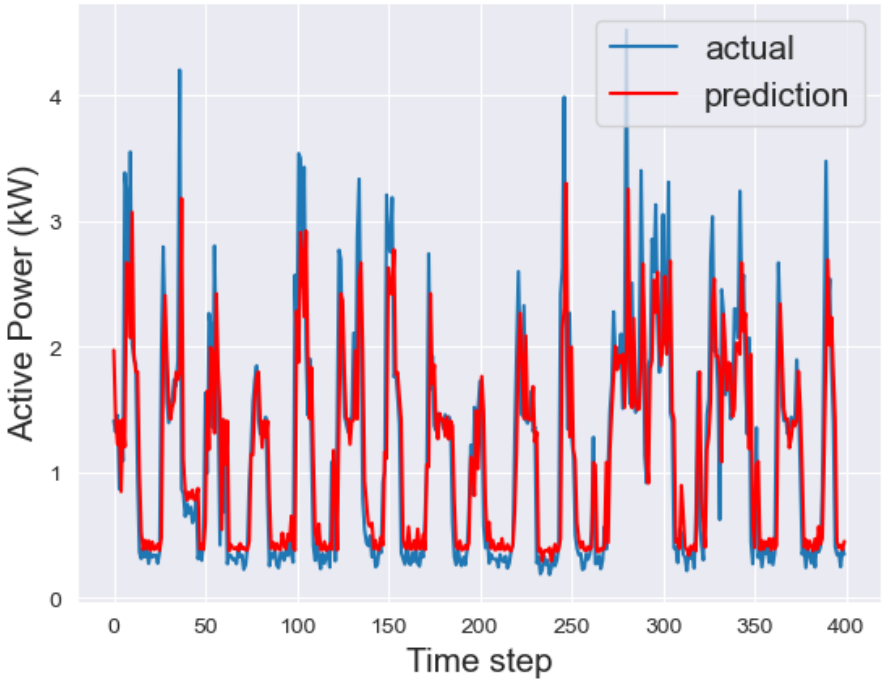

Figure 3.1: RandomForestRegressor training results

On the IoT device, power readings must be scaled and quantized before inference. The model prediction, along with the most recent readings, is sent to the cloud application. Additionally, the prediction is used to determine the power modality, based on two thresholds: `LOW_TH` and `MAX_TH`. These thresholds help classify power usage into three modes, allowing the system to avoid high power peaks that could destabilize the electrical system and to reduce energy costs.

3.1.1 LEDs and Button for Energy Saving Modes

The button on the building router node is used to activate or deactivate the energy-saving mode. The LEDs on both the building router and floor sensor nodes indicate the current operating mode: green, blue, and red, representing increasing levels of energy-saving severity.

3.2 Battery Setpoint Control

The battery setpoint is calculated on the cloud platform and sent back to the IoT border router. This choice was necessary because the energy price must be retrieved from the internet. Since the data required

for this computation was already available in the cloud application for logging purposes, the calculation is offloaded from the constrained device.
In particular, the battery setpoint is computed in kilowatts (kW)—indicating the power to charge the battery—based on the following three factors:

- **Energy price:** The lower the price, the more energy we aim to store for high-demand periods.
- **Prediction deviation:** The greater the difference between the predicted and most recent power readings, the more we discharge the battery to avoid problematic power peaks in the grid.
- **State of Charge (SOC):** The latest SOC value is used to correct the final result, avoiding overcharging above 90% and discharging below 5%, thereby maintaining battery health.

3.3 AC and Window Setpoint Control Logic

To regulate temperature and lighting within the smart environment, we designed two control algorithms executed directly on the IoT device: one for adjusting the AC temperature setpoint and another for controlling the window position setpoint.

Temperature Setpoint Adjustment

The AC control logic computes a new temperature setpoint based on the current room temperature, the desired temperature (`temp_required`), and the energy modality parameter (`modality`).
The algorithm operates as follows:

- It computes the difference between the desired and current room temperature.
- A scaling factor is selected depending on the current power modality: `4.0` for low, `5.0` for medium, and `8.0` for high energy-saving modes.
- The setpoint is updated by adjusting it proportionally to the temperature difference and the scaling factor.
- To avoid abrupt temperature changes, the new setpoint is clamped within $\pm 2^{\circ}$C of the target temperature.

This strategy ensures smoother temperature transitions and prevents sudden shifts that could increase energy usage or reduce user comfort.

Window Openness Setpoint Adjustment

Lighting control is managed by adjusting the openness of smart windows. The algorithm modifies the window setpoint to approach the desired ambient light level using a simple proportional feedback mechanism.
Specifically, the difference between the desired and current light levels is scaled and subtracted from the current setpoint. The resulting value is then clamped between `0.0` (fully closed) and `100.0` (fully open). This guarantees that the system responds gradually to changing light conditions while maintaining a valid physical state for the window actuator.

4 Cloud Application

4.1 Database Schema

The MySQL database contains the following tables:

Table: `battery`

Column	Type	Description
`id`	`INT (AUTO_INCREMENT)`	Unique identifier for each record.
`timestamp`	`TIMESTAMP`	Time when the data was recorded in the database.
`value`	`FLOAT`	State of charge (SOC) of the battery, in percent.

Table: `light`

Column	Type	Description
`id`	`INT (AUTO_INCREMENT)`	Unique identifier for each record.
`timestamp`	`TIMESTAMP`	Time when the data was recorded in the database.
`value`	`FLOAT`	Measured light level (lumens).
`floor`	`INT`	Floor on which the light measurement was taken.

Table: `power`

Column	Type	Description
`id`	`INT (AUTO_INCREMENT)`	Unique identifier for each record.
`timestamp`	`TIMESTAMP`	Time when the data was recorded in the database.
`value_active`	`FLOAT`	Measured active power (kW).
`value_reactive`	`FLOAT`	Measured reactive power (kW).

Table: `temperature`

Column	Type	Description
`id`	`INT (AUTO_INCREMENT)`	Unique identifier for each record.
`timestamp`	`TIMESTAMP`	Time when the data was recorded in the database.
`value`	`FLOAT`	Measured temperature value (°C).
`floor`	`INT`	Floor on which the temperature was measured.

4.2 Grafana

We connect the MySQL database to Grafana and created the following interface (Figure 4.1).

4.3 CLI

The CLI provides full control over the cloud application. Specifically, it allows for the management of air conditioning, windows, light levels, and temperature setpoints per floor. Users can enable or disable

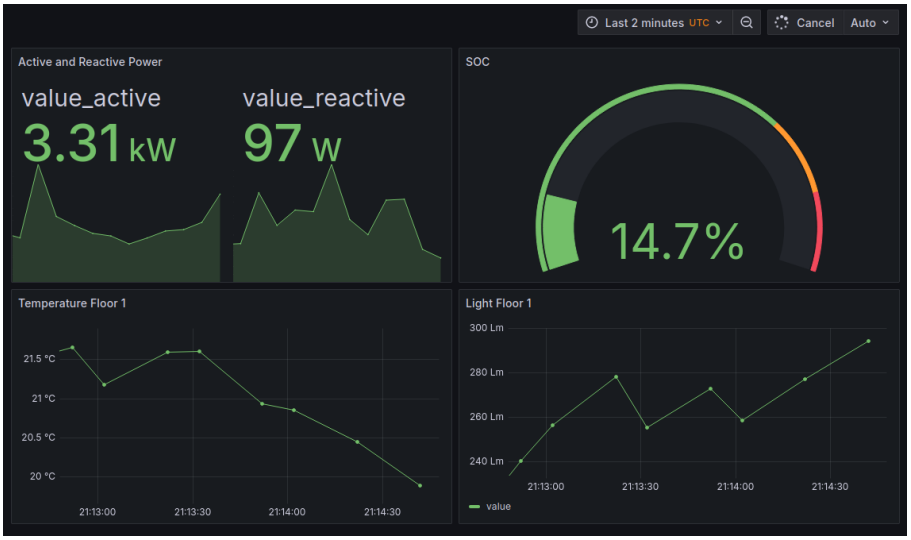

Figure 4.1: Grafana Interface

the automatic control of the air conditioning and windows for each floor. Additionally, battery input power can be configured. The CLI also provides access to recent logs and stored data from the MySQL database.

Listing 4.1: CoAP Client Manager Menu

```
=== CoAP Client Manager ===
Available floors: [0, 1]
1. Send AC Command
2. Send Window Command
3. Send Temperature Command
4. Send Light Command
5. Send Battery Command
6. Enable/Disable Dynamic Control of Floor Sensors
7. View Stored Data
8. View Logs
9. Enable/Disable automatic SOC control
10. Exit
Choose an option (1-9)
```